

EDR Mastery Roadmap

(Post-Lab)

A structured progression for analysts
who investigate behavior, not alerts

Endpoint

Raw Telemetry

Behavioral Analysis

Response

- explorer.exe → chrome.exe > powershell.exe
- event.code: 4688
- event.category: process

INTRODUCTION

This roadmap exists for **analysts who already have endpoint telemetry flowing** and want to move beyond tool usage into **true EDR competence**.

It assumes you have completed at least one real Elastic EDR lab and are no longer blocked by installation, agent connectivity, or basic navigation. From this point forward, the problem is **not tooling** — it is **analyst reasoning**.

Most SOC failures do not happen because the EDR missed something. They happen because analysts:

- trusted alerts instead of evidence
- misclassified normal OS behavior as threats
- failed to explain what happened without a rule name
- escalated noise and missed intent

This roadmap is designed to **eliminate those failure modes**.

What this roadmap is (and is not)

This is **not**:

- a detection rule collection
- a malware lab
- a MITRE checklist
- a dashboard-driven workflow

This **is**:

- a structured progression from raw telemetry to analyst judgment
- a discipline for classifying reality from endpoint data
- a method to reason about attacker behavior *before* detections exist

- a way to become useful during real incident response

If you are looking for “EDR alerts explained,” this document is not for you.

Hard assumptions (stop if any are false)

You **must** be able to answer yes to all of the following:

- You see **continuous endpoint telemetry** in Discover
- You can explain **process, file, and network events** from raw fields
- You understand **Fleet is the control plane**, not the detection engine

If any of these are false, **do not continue**.

Fix ingestion, visibility, or fundamentals first.

Proceeding without these guarantees produces false confidence.

Core philosophy

This roadmap enforces one principle throughout:

You do not investigate alerts. You investigate behavior.

Endpoint telemetry must be treated with the same seriousness as:

- packet captures (PCAP)
- syscall traces
- kernel audit logs

Alerts, severities, and “EDR verdicts” are **secondary artifacts**, not sources of truth.

If you rely on:

- “Elastic says it’s clean”

- rule names
- alert confidence scores

you are not practicing telemetry ownership.

How progression works

The roadmap is deliberately staged. Each stage builds a **non-skippable skill**:

- **Stage 1 — Telemetry Ownership**
Learn to explain *why* processes, files, and network events exist — especially when they look boring.
- **Stage 2 — Behavioral Thinking**
Stop reacting to alerts. Start forming hypotheses from deviations, context, and intent.
- **Stage 3 — Controlled Attack Simulation**
Generate small, explainable attacker-like behaviors and reconstruct them *only from raw events*.
- **Stage 4 — Detection Engineering (optional)**
Learn how detections are built from analyst understanding, not copied from repositories.
- **Stage 5 — Response Discipline**
Learn when **not** to isolate, block, or escalate.
- **Stage 6 — Failure & Debugging**
Become the analyst who fixes broken telemetry instead of reinstalling agents.

Each stage has **exit criteria**.

If you hesitate, guess, or rely on alerts, you have not passed.

Expected outcome

By completing this roadmap correctly, you should be able to:

- Instantly distinguish:
 - normal OS behavior
 - admin activity
 - suspicious but explainable actions
 - genuinely malicious chains
- Reconstruct endpoint activity **from raw events alone**
- Explain behavior clearly to:
 - senior analysts
 - incident responders
 - detection engineers
- Build fewer detections — but ones that actually work
- Avoid over-response, alert fatigue, and false incidents

This is what **real SOC maturity** looks like.

STAGE 1 — TELEMETRY OWNERSHIP (FOUNDATION)

Core mindset (non-negotiable)

If you rely on **alerts, dashboards, or severity labels**, you are not doing telemetry ownership.

You are **manually classifying reality from raw events**.

Treat endpoint telemetry exactly like:

- packet captures (PCAP)
- system call traces
- kernel logs

No alerts. No rules. No “EDR says it’s clean”.

1. PROCESS LINEAGE DRILLS (MOST IMPORTANT)

You are not analyzing *processes*.

You are analyzing **chains of intent**.

What to pull (Elastic example)

Query only raw process events:

```
event.category:process AND event.type:start
```

pick **1 event**. Do NOT cherry-pick suspicious ones.

For EACH process, reconstruct this manually

A. Parent → Child → Grandchild

Example (realistic):

```
explorer.exe
├── chrome.exe
│   └── powershell.exe
```

Immediately ask:

- Why did **chrome.exe** spawn **powershell.exe**?
- Is this user-driven, extension-driven, or exploit-driven?

B. User context (this decides 50% of verdict)

Check:

- user.name
- user.domain
- user.id
- process.integrity_level
- process.session_id

Classify the context:

- Interactive user
- Admin user
- SYSTEM
- Service account

Example reasoning:

- powershell.exe as **SYSTEM** → normal for updates, suspicious for browsers
- powershell.exe as **user** → scripts, dev tools, or malware

C. Command line intent (critical skill)

You must read command lines **like code**, not strings.

Bad analysts read:

powershell.exe -enc JABXAE...

Good analysts translate:

- Encoded PowerShell
- Why encoded?
- What spawned it?

- Was execution policy bypassed?

Example translation:

powershell.exe -nop -w hidden -enc ...

→ Intent: run hidden PowerShell without profile, likely automation or evasion

D. Write 1–2 lines explaining WHY IT EXISTS

This is mandatory.

Bad:

"PowerShell executed"

Good:

"Chrome spawned PowerShell under user context likely due to extension-initiated automation; encoded command suggests obfuscation but no lateral behavior observed."

If you cannot explain *why*, you do not understand the event.

elastic Find apps, content, and more. AI Assistant

Discover Check out context-aware Discover Try ES|QL Inspect Alerts + Save

Untitled +

Data view All logs event.category:process AND Last 15 minutes

Search field names 0

Available fields 324

- @timestamp
- agent.ephemeral_id
- agent.id
- agent.name
- agent.type
- agent.version
- data_stream.dataset
- data_stream.namespace
- data_stream.type
- destination.address

Add a field

Auto interval No breakdown

Documents (580) Field statistics

Sort fields 1

Summary

event.type start @timestamp Feb 4, 2026 @ 11:51:22.596...

event.category process

Rows per page: 100

Document 2 of 500

Actions

Table JSON

Search field names or values 0

Selected only

Field	Value
_id	AZwnUFFV7EvSvdVCSz_I
_ignored	(null)
_index	.ds-logs-system.security-default-2026.02.02-000001
_score	(null)
@timestamp	Feb 4, 2026 @ 11:51:22.596
agent.ephemeral_id	a8ca140f-e26c-4a23-b779-2f84cd04214

Rows per page: 25

JSON

Copy to clipboard

```
{
  "parent": {
    "name": "services.exe",
    "pid": 684,
    "executable": "C:\\Windows\\System32\\services.exe"
  },
  "name": "svchost.exe",
  "pid": 11088,
  "executable": "C:\\Windows\\System32\\svchost.exe",
  "command_line": "C:\\Windows\\system32\\svchost.exe -k LocalSystemNetworkRestricted -s WPDBusEnum"
}
```

What this event actually is (before judgment)

```
Event type: process start
Process: svchost.exe
Host: loop
User: NT AUTHORITY\SYSTEM
Time: 2026-02-04T06:21:22Z
```

This is **not** a user-launched process. This is **service infrastructure**.

If you treat this like “just another svchost”, you fail Stage 1.

You must prove *why* it exists.

1. Process Lineage Reconstruction (MANDATORY)

Parent → Child

```
services.exe (PID 684)
├── svchost.exe (PID 11088)
```

No grandchild yet (this is only the start event).

Parent validation

- services.exe
- Path: C:\Windows\System32\services.exe
- Signed by: Microsoft Windows Publisher
- Trusted: true

This is the **Windows Service Control Manager**.

If svchost.exe does **not** originate from services.exe, that is a red flag.

Here it **does**.

2. Command Line Intent (THIS IS WHERE ANALYSTS FAIL)

```
svchost.exe -k LocalSystemNetworkRestricted -s WPDBusEnum
```

Break it down:

-k LocalSystemNetworkRestricted

- Service group running as **SYSTEM**
- Network access restricted (not arbitrary outbound)
- Common for OS components

-s WPDBusEnum

This is the **service name**, not an argument.

WPDBusEnum = *Portable Device Enumerator Service*

Used for:

- USB devices
- MTP devices (phones, cameras)
- Hardware discovery

This is **hardware-related OS behavior**, not user behavior.

If you didn't recognize WPDBusEnum, that's a knowledge gap, not a threat.

3. User & Token Context (DECISION-DRIVER)

- user.name: SYSTEM
- integrity_level: system
- authentication_id: 0x3e7 (LocalSystem)
- session_id: 0

This confirms:

- Non-interactive
- Boot / service lifecycle related
- Not user-triggered malware

If this were running under a **real user SID**, it would be abnormal.

4. Binary Trust & Location (BASELINE CHECK)

Binary

- Path: C:\Windows\System32\svchost.exe
- Original file name: svchost.exe
- SHA256 present
- Signed by Microsoft
- Trusted: true

Exploit check you *should* be thinking about

Malware abuses svchost.exe by:

- Running from **non-System32 paths**
- Injecting into an existing svchost
- Launching svchost with **non-service arguments**

None of that is present here.

5. Suspicious-looking fields (DO NOT PANIC)

created_suspended: true

This is **normal for service-hosted processes**.

Windows often:

1. Creates process suspended
2. Applies mitigations
3. Attaches service DLL
4. Resumes execution

This is **not** malware behavior by itself.

If you panic on this flag, you are alert-driven, not telemetry-driven.

6. Mitigations Applied (VERY IMPORTANT SIGNAL)

"CET dynamic APIs can only be called out of proc"

"CF Guard"

These are **modern Windows exploit mitigations**:

- Control Flow Guard
- CET protections

Malware does **not** opt into these willingly.

This reinforces:

- OS-managed process
 - Not attacker-crafted
-

7. What you should mentally ask

Even though this looks clean, a real analyst asks:

1. Did this svchost:
 - Spawn child processes?
 - Load unexpected DLLs?
2. Did it initiate **unexpected outbound network connections**?
3. Did it write files outside:
 - System32
 - WinSxS
 - Driver or device paths?

That's how you **confirm**, not assume.

8. Final Classification (NO HEDGING)

Verdict

 **Normal admin / OS activity**

One-line explanation:

"SYSTEM-level svchost.exe launched by services.exe to host the WPDBusEnum service, consistent with normal Windows device enumeration behavior."

2. FILE ACTIVITY CORRELATION (INTENT VALIDATION)

File writes are **evidence of intent**, not inherently malicious.

Pull raw file events

`event.category:file`

Pick file writes and **trace backwards**.

Correlate file writes to ONE of these parents

A. Installer activity (usually valid)

Typical parents:

- `msiexec.exe`
- `setup.exe`
- `update.exe`

Ask:

- Install directory?
- Signed binary?
- Expected file types?

Normal:

msiexec.exe → writes DLLs to Program Files

Suspicious:

msiexec.exe → writes EXE to AppData\Roaming

B. Browser downloads

Typical parents:

- chrome.exe
- msedge.exe
- firefox.exe

Check:

- Download directory
- File extension
- Follow-on execution

Normal:

chrome.exe → Downloads → .pdf

Suspicious:

chrome.exe → AppData\Local\Temp → .exe → executed

C. Script execution

Parents:

- powershell.exe
- wscript.exe
- cscript.exe
- python.exe

Ask one question:

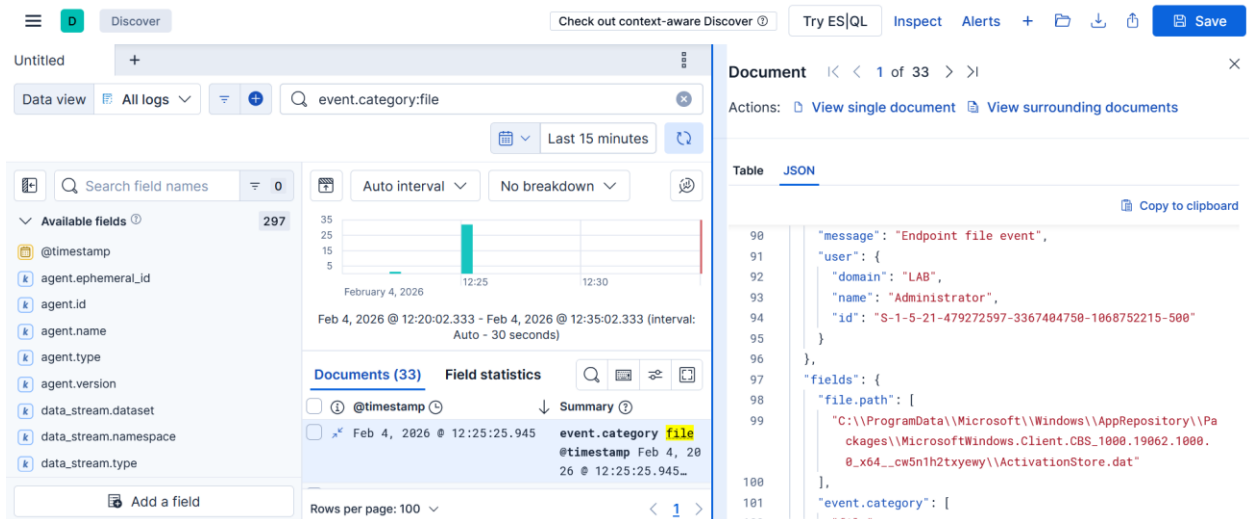
Is this file write justified by the parent's role?

Example:

- PowerShell writing .ps1 → normal
- PowerShell writing .dll → high scrutiny
- PowerShell writing .exe → red flag

Rule you must internalize

A file write without a **clear business reason** is suspicious, even if clean.



Identify the event (no interpretation yet)

- **Event type:** file → deletion

- **File:** ActivationStore.dat
- **Path:**
 - C:\ProgramData\Microsoft\Windows\AppRepository\Packages\MicrosoftWindows.Client.CBS_1000.19062.1000.0_x64__cw5n1h2txyewy\
 - ActivationStore.dat
- **Process:** sihost.exe
- **Process path:** C:\Windows\System32\sihost.exe
- **User:** LAB\Administrator
- **Time:** 2026-02-04T06:55:25Z

This is **not a file creation, not execution, not persistence**.
It is a **state change** (deletion).

1. Process legitimacy (first gate)

Process: sihost.exe

- Binary path: System32
- Signed by: Microsoft Windows
- Trusted: true

sihost.exe = **Shell Infrastructure Host**

Its job:

- Windows shell lifecycle
- AppX / UWP infrastructure
- User session UI plumbing
- Start menu, app repository interactions

If sihost.exe were:

- Running from user space ✖
- Unsigned ✖
- Spawning scripts ✖

→ then panic.

Here, it passes baseline.

2. User context (this matters)

- User: Administrator
- Domain: LAB
- Not SYSTEM
- Interactive admin session

Important nuance:

- **Admin user ≠ malware**
- **Admin user ≠ safe**

But sihost.exe is **expected** to operate under logged-in user sessions.

If this deletion happened under SYSTEM, it would still be fine.

If it happened under a **low-privileged user** writing here → investigate.

3. File path analysis (this is where intent lives)

Path breakdown

C:\ProgramData\Microsoft\Windows\AppRepository\Packages\
MicrosoftWindows.Client.CBS_...

This directory is:

- Owned by Windows
- Used by **UWP / AppX package management**
- Frequently rewritten, rebuilt, cleaned

File: ActivationStore.dat

Key facts:

- .dat → data file, not executable
- Located inside a **package versioned directory**
- Not a startup location
- Not user-writable by default apps

This is **state**, not payload.

4. Action analysis: deletion ≠ destruction

This is where junior analysts fail.

Ask:

Why would Windows delete its own .dat file?

Common legitimate reasons:

- AppX package update
- Store app refresh
- Cache invalidation
- User session reload
- Version rollback

Windows routinely:

- Deletes
- Recreates

- Renames
these files during normal operation.
-

5. What is *not* present (very important)

No signs of abuse:

- **✗** No suspicious parent (PowerShell, cmd, wmic)
- **✗** No execution from the file
- **✗** No rename-to-exe
- **✗** No temp directory abuse
- **✗** No persistence path involved
- **✗** No unsigned binary touching the file

Also:

- file.entropy: 0 → not packed, not encrypted payload
 - file.size: -1 → deletion event, size unknown (expected)
-

6. Correlation logic (Stage-1 skill check)

Answer this **one question**:

Is this file deletion justified by the parent process's role?

Parent role:

sihost.exe manages shell + app repository state.

File role:

AppRepository activation data.

Verdict:

Yes. Fully justified.

If this were:

- powershell.exe deleting this → investigate
- chrome.exe deleting this → investigate
- Random EXE deleting this → high risk

Context alignment is everything.

7. Final classification (no hedging)

Category

 **Normal admin / OS behavior**

One-line analyst explanation (you must be able to write this):

"Signed Windows shell process deleted an AppRepository activation data file as part of normal UWP package or session lifecycle; no execution or persistence indicators observed."

3. NETWORK SANITY CHECKS (BASELINE OR DIE)

You are not threat hunting yet.
You are building a **mental baseline**.

Pull network events

`event.category:network AND event.type:connection`

You MUST be able to instantly recognize these

A. Browser traffic

Characteristics:

- Destination: CDN, cloud providers
- Ports: 443
- Process: browser binary

If browser traffic spawns **child processes or scripts**, escalate.

B. Update services

Examples:

- OS updates
- EDR updates
- Browser updates

Traits:

- SYSTEM context
- Known vendor domains
- Regular timing

C. Internal OS chatter

Includes:

- Localhost traffic
- RPC
- SMB
- DNS

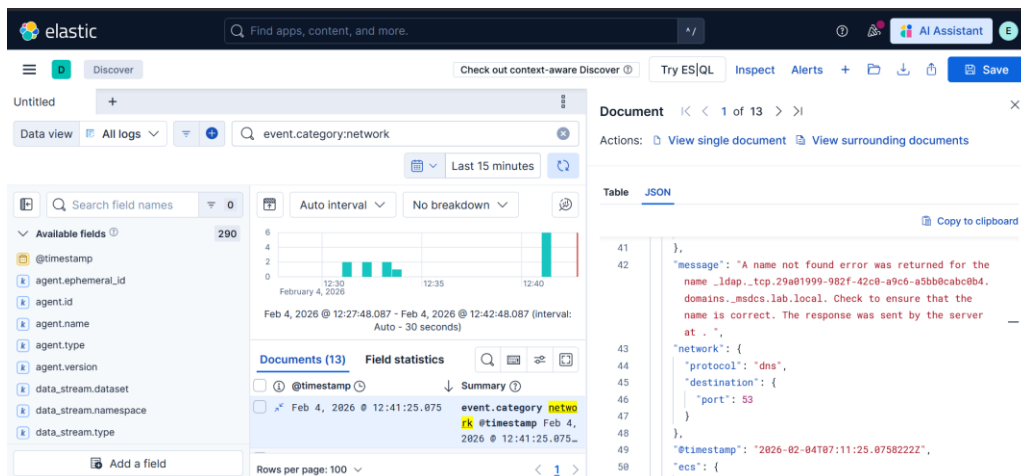
If you don't recognize this traffic, you lack OS fundamentals.

Anything unexplained → investigate backward

Start from:

- Process
- Parent
- User
- Command line

Never start from destination IP.



First I assumed that its an LDAP authentication error , but it isn't

This is **not an LDAP authentication error**. It is a **DNS SRV lookup failure during AD service discovery**. That distinction matters.

I'll break it down strictly, Stage-1 style.

0. What this event is (no guessing)

- **Protocol:** DNS
- **Destination port:** 53
- **Process:** svchost.exe
- **User:** NT AUTHORITY\NETWORK SERVICE
- **Outcome:** failure
- **DNS query:**
_ldap._tcp.29a01999-982f-42c0-a9c6-a5bb0cabcb0b4.domains._msdcs.lab.local

This is a **DNS lookup_result failure**, not an authentication attempt.

1. Process & context validation (baseline)

Process

- svchost.exe
- Signed by Microsoft
- Running as **NETWORK SERVICE**

This context is **expected** for:

- Netlogon
- DNS Client
- AD-related background services

If this were:

- powershell.exe ❌
- A user-launched binary ❌
- Unsigned ❌

Then we would escalate. Here, we don't.

2. Decode the DNS name (this is the key skill)

Query:

_ldap._tcp.<GUID>.domains._msdcs.lab.local

Break it down:

- _ldap._tcp
→ SRV record for **LDAP over TCP**

- <GUID>
→ **Domain Controller object GUID**
- _msdcs.lab.local
→ Microsoft Domain Controller Service DNS zone

This lookup is used for:

- Domain Controller discovery
- Replication checks
- Secure channel validation
- Computer account maintenance

 Important:

This happens **before** authentication. No credentials are exchanged here.

3. Why your “LDAP authentication error” assumption is incorrect

LDAP authentication errors happen **after**:

- A DC is located
- LDAP session is established
- Bind is attempted

This event shows:

- DNS SRV lookup
- Name not found
- No resolved IPs
- No LDAP traffic (389/636)

So the failure is **name resolution**, not auth.

Correct mental model:

DNS discovery → LDAP connection → LDAP bind (auth)

This failed at **step 1**.

4. Why this failure is usually NORMAL

Common legitimate reasons:

1. **Stale AD metadata**
 - Old DC GUID still referenced
 - DC decommissioned incorrectly
2. **Lab / non-production domain**
 - Partial DNS records
 - Missing _msdcs entries
3. **Startup / background retries**
 - Windows periodically retries DC discovery
 - Failures are expected and noisy
4. **Single-DC or broken DNS**
 - No SRV record exists
 - DNS still queried anyway

If this were a real auth problem, you would see:

- Kerberos failures
- Event ID 4625 / 4768
- LDAP bind errors
- TCP 389/636 attempts

None are present.

5. Security posture assessment (Stage-1 classification)

Ask the only question that matters:

Is this network activity justified by the process role and user context?

- Process: Windows service host
- User: NETWORK SERVICE
- Destination: DNS
- Query: AD SRV record

✓ Fully justified.

No C2 indicators

No DGA

No suspicious domain

No user-triggered behavior

6. Correct classification

Category

✓ **Normal OS / domain service behavior**

Correct one-line explanation

"Windows service attempted DNS SRV lookup for LDAP domain controller discovery under NETWORK SERVICE context; lookup failed due to missing or stale AD DNS record, not authentication."

EXIT CRITERIA (THIS IS A HARD GATE)

You are **NOT allowed** to move to detections until this is automatic.

You must look at **raw telemetry** and instantly classify:

1. **Normal user behavior**
 - Browsing, document access, app launches
2. **Admin activity**
 - SYSTEM processes, installers, config tools
3. **Suspicious but explainable**
 - Encoded PowerShell with known tooling
 - Automation scripts
 - Dev environments
4. **Genuinely anomalous**
 - Browser → script → binary drop
 - User context doing SYSTEM-like behavior
 - Unsigned binaries executing from user paths

If you **hesitate**, that is a failure signal — not learning.

STAGE 2 — BEHAVIORAL THINKING (CORE EDR SKILL)

What this stage actually means (strip the buzzwords)

Most junior analysts do this:

Alert fired → read rule → decide benign/malicious

That is **signature thinking**.

It fails the moment:

- the rule is noisy

- the attacker changes technique
- the alert doesn't fire

Behavioral thinking reverses the flow:

Observed behavior → context → intent → risk

Alerts become **evidence**, not the starting point.

Mental model you must enforce

Every investigation starts with a **behavioral hypothesis**.

"A process did X, in context Y. That is unusual because Z."

Only **after** that do you:

- check alerts
- check detections
- check threat intel

If you skip this, you are not an EDR analyst — you're a dashboard reader.

The 3 mandatory questions (non-negotiable)

For **every behavior**, answer these **before touching any rule**:

1. What is unusual?

Concrete deviation. No vibes.

Bad:

"Looks suspicious"

Good:

"powershell.exe launched with encoded command by winword.exe"

2. In what context would this be normal?

If you cannot explain normality, you cannot detect abuse.

Example:

- Admin automation
 - Software updater
 - IT script
 - Dev workstation
-

3. What would make it malicious?

This is **intent**.

Indicators:

- User-writable paths
 - Unexpected parent
 - Encoded/obfuscated arguments
 - Network + execution chaining
 - Time-of-day anomalies
-

Behaviors to study (your list, expanded correctly)

1. Command-line abuse

What this really is

Attackers rarely invent binaries. They **abuse arguments**.

Behavioral signals

- -enc, -nop, -w hidden
- Long base64 strings
- Chained commands (;, &&)
- Output redirection to temp paths

Normal context

- Admin scripts
- Build pipelines
- Scheduled maintenance

Malicious context

- Spawned by Office, browser, PDF reader
 - Runs from user profile
 - Executes immediately after file open
-

2. Parent-child mismatches

Core idea

Processes have *expected parents*.

Examples

- winword.exe → powershell.exe ❌
- chrome.exe → cmd.exe ❌
- services.exe → notepad.exe ❌

Normal context

- explorer.exe → notepad.exe

- services.exe → svchost.exe

Malicious amplification

- Parent is user-facing app
 - Child is scripting engine
 - Followed by network or persistence
-

3. Execution from user-writable paths

Why attackers love these

- No admin rights needed
- AV exclusions often weaker
- Blend with legitimate clutter

Paths to memorize

- Downloads
- AppData\Local\Temp
- AppData\Roaming
- Desktop

Normal context

- Installers
- Self-updating apps

Malicious context

- Random names
 - Immediate execution
 - No installer logic
-

4. Living-off-the-Land Binaries (LOLBins)

Reality check

LOLBins are not malicious.

Context is malicious.

Examples

- certutil.exe
- mshta.exe
- rundll32.exe
- wmic.exe

Normal

- Admin tasks
- Legacy apps

Malicious

- Used by user apps
 - Pulling remote content
 - Writing to temp + executing
-

5. Credential access precursors (pre-dump)

You are **not dumping yet**.

You are watching the setup.

Behaviors

- LSASS handle access attempts
- whoami /all
- net user, net localgroup
- Token enumeration

Normal

- IT troubleshooting

- Domain admin activity

Malicious

- Standard user context
- Followed by lateral movement
- Occurs after initial access

One SIMPLE practical (Elastic Security)

This is intentionally basic.

The goal is **thinking**, not hunting wizardry.

Practical: Command-Line Behavioral Hypothesis

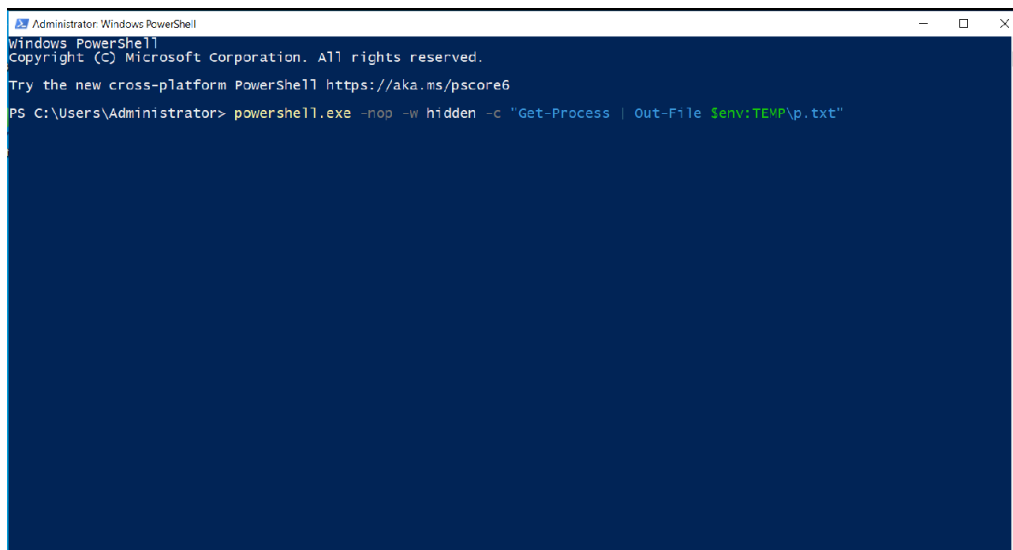
Platform: Elastic Security (Endpoint data)

Step 1: Generate a benign but “abusable” behavior

On a Windows endpoint with Elastic Agent:

Open **PowerShell** and run:

```
powershell.exe -nop -w hidden -c "Get-Process | Out-File $env:TEMP\p.txt"
```



```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/powershell

PS C:\Users\Administrator> powershell.exe -nop -w hidden -c "Get-Process | Out-File $env:TEMP\p.txt"
```

What it does (literal)

1. **powershell.exe**
Launches a new PowerShell process.
2. **-nop (NoProfile)**
Skips loading user/system PowerShell profiles.
Effect: faster start, fewer artifacts, avoids profile-based logging or prompts.
3. **-w hidden (WindowStyle Hidden)**
Runs with no visible console window.
Effect: user doesn't see anything.
4. **-c (Command)**
Executes the quoted command and exits.
5. **Get-Process**
Enumerates all running processes (names, PIDs, CPU, memory, etc.).
6. **Out-File \$env:TEMP\p.txt**
Writes the output to a file named p.txt in the current user's %TEMP% directory.

This is **not malware** — that's the point.

Step 2: Observe (no alerts yet)

Go to **Security** → **Discover / Timeline**

Filter on:

- process.name : "powershell.exe"

Look at:

- process.command_line
 - process.parent.name
 - process.executable
 - process.working_directory
-

Step 3: Answer the 3 questions (write this down)

What is unusual?

- Hidden window
- No profile
- Non-interactive execution

When could this be normal?

- Admin automation
- Background maintenance script

What would make this malicious?

- Parent = winword.exe OR chrome.exe
- Encoded command
- Network call before/after
- Output written then executed

If you can't answer these cleanly, you are not ready for detection engineering.

Step 4: Optional sanity check (NOT a rule)

Search for:

- Parent = explorer.exe
- User = your account
- No network activity

Conclusion:

Behavior is **low risk in this context**, but **highly abusable**

That's behavioral reasoning.

What you should NOT do yet

- ❌ Write a detection rule
- ❌ Tune alerts
- ❌ Add exceptions
- ❌ Chase threat intel

Those come **after** hypothesis maturity.

Behavioral Analysis of the Event (No Alerts, No Rules)

Data source: Windows Security Event **4688** ingested into Elastic

Behavior under study: Command-line abuse

elastic

Find apps, content, and more.

AI Assistant

Discover

Check out context-aware Discover

Try ES|QL

Inspect

Alerts

Save

Untitled

process.name:"powershell.exe"

Last 15 minutes

Available fields 332

@timestamp

agent.ephemeral_id

agent.id

agent.name

agent.type

agent.version

data_stream.dataset

data_stream.namespace

data_stream.type

Add a field

Auto interval

No breakdown

Feb 6, 2026 @ 10:41:13.374 - Feb 6, 2026 @ 10:56:13.374 (interval: Auto - 30 seconds)

Documents (218)

Field statistics

Sort fields

@timestamp

Feb 6, 2026 @ 10:55:46.149

process.name powershell.exe

Rows per page: 100

200

150

100

50

0

February 6, 2026

10:45

10:50

10:55

Document

1 of 218

View single document

View surrounding documents

Table

JSON

Search field names or values

Selected only

Field

Value

event.category

process

event.code

4688

event.created

Feb 6, 2026 @ 10:55:47.667

event.dataset

system.security

event.ingested

Feb 6, 2026 @ 10:55:53.000

event.kind

event

Rows per page: 25

1 2 3 4

Document

1 of 218

View single document

View surrounding documents

Table

JSON

Search field names or values

Selected only

Field

Value

process.args

["C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe", -nop, -w, hidden, -c, "Get-Process | Out-File C:\Users\ADMINI~1\AppData\Local\Temp\p.txt"]

process.command_line

"C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe" -nop -w hidden -c "Get-Process | Out-File C:\Users\ADMINI~1\AppData\Local\Temp\p.txt"

process.executable

C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

process.name

powershell.exe

process.parent.executable

C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe

Rows per page: 25

1 2 3 4

1 What behavior occurred? (FACTS ONLY)

A new process was created:

- **Process:** powershell.exe
- **Parent:** powershell.exe
- **User:** Administrator (built-in)
- **Command line:**
powershell.exe -nop -w hidden -c "Get-Process | Out-File C:\Users\ADMINI~1\AppData\Local\Temp\p.txt"
- **Output location:** User-writable temp directory
- **Window:** Hidden
- **Profile loading:** Disabled (-nop)
- **Token:** Full token (Type 1 – built-in Administrator)

This is **not an opinion**. This is what happened.

2 What is unusual? (BEHAVIORAL DEVIATION)

You must isolate **deviations**, not keywords.

Unusual elements

Element	Why it matters
-w hidden	Explicit UI suppression
-nop	Bypasses PowerShell profile logging/customizations
Output to %TEMP%	Common staging location
Self-spawn (powershell → powershell)	Scripted or chained execution
Process enumeration	Reconnaissance precursor

⚠ **Important correction:**

Nothing here is *intrinsically malicious*.

But **multiple abuse-prone traits are present simultaneously**.

That combination is the signal.

3 In what context would this be normal?

If you can't justify normality, you are guessing.

Legitimate contexts

- Administrator running:
 - background scripts
 - automation
 - troubleshooting commands
- Lab / testing environment
- Headless execution (no user interaction needed)

Your environment context supports this:

- Host name: loop.lab.local
- User: built-in Administrator
- No network activity
- No child execution afterward

So far: **benign-leaning**.

4 What would make this malicious? (INTENT SHIFT)

This is the most important part.

The **exact same command** becomes malicious if **context changes**.

Escalation conditions

Change	Why it flips
Parent = winword.exe / chrome.exe	Initial access vector
-enc or long Base64	Obfuscation
Followed by Invoke-WebRequest	Payload delivery
Temp output → execution	Staging → execution chain
Non-admin user	Privilege boundary crossing
Lateral movement after	Post-recon exploitation

⚠ Key analyst insight

This event is **Stage-0 reconnaissance** behavior.

On its own: low risk.

In sequence: dangerous.

5 What this event is NOT

Be precise.

- ❌ Not malware
- ❌ Not an IOC
- ❌ Not an alert by itself
- ❌ Not a detection rule candidate (yet)

It is **behavioral evidence**.

6 Correct Behavioral Verdict

Classification:

Benign execution with high abuse potential

Risk posture:

Low *in isolation*, high *in adversary chains*

This is the correct analyst answer.

Anything else is either:

- rule-driven thinking
- keyword paranoia
- or alert addiction

One mistake you must NOT make

Do **not** say:

"This is fine because it's Administrator"

Attackers **love** Administrator contexts.

Privilege ≠ trust.

STAGE 3 — CONTROLLED ATTACK SIMULATION

Concept (what this stage really is)

This stage is **behavioral validation under controlled conditions**.

You are not:

- testing Elastic detections
- running malware
- proving compromise

You **are**:

- generating *small, explainable attacker-like deviations*
- validating whether you can **reconstruct intent purely from raw telemetry**
- training your **investigation muscle**, not your tooling muscle

If you rely on alerts here, you fail the stage.

Rationale (why this stage exists)

SOC analysts fail in three predictable ways:

1. **Alert dependence**
→ "No alert = no incident"
2. **Signature bias**
→ "This matches a known rule, therefore malicious"
3. **Narrative collapse**
→ Cannot explain *what happened* without the rule text

STAGE 3 exists to break all three.

If you cannot explain:

- *what happened*
- *in what order*
- *why it matters*

from raw events, you are not an EDR analyst.

Real-world Elastic Security context

In production Elastic environments:

- Alerts are **lagging indicators**
- Detections are **imperfect**
- Attackers operate **below thresholds**
- Analysts investigate **raw data first** during IR

Elastic gives you:

- endpoint telemetry
- Windows Security events
- process ancestry
- command-line visibility

STAGE 3 teaches you to **use those correctly**.

The hard rule (non-negotiable)

If you can't explain it from raw events, don't run it.

This rule enforces:

- intent clarity
- scope control
- analyst discipline

Running noisy payloads at this stage **destroys learning**.

What you are allowed to simulate (and why)

These are **micro-deviations**, not attacks.

Simulation	Why it's allowed
Encoded PowerShell	Tests obfuscation reasoning
cmd → powershell	Tests parent-child logic
Script from Downloads	Tests execution context
LOLBins with flags	Tests intent vs utility

Each deviation is:

- explainable
 - reversible
 - observable in raw logs
-

Required workflow (do not reorder)

1. Generate activity
2. Go to Discover
3. Identify raw events
4. Reconstruct the story
5. Decide:
 - Benign
 - Suspicious
 - Malicious

Alerts come last, not first.

END-TO-END PRACTICAL (Elastic Security)

Simulation: cmd → encoded PowerShell

This single exercise touches **three attacker behaviors**:

- parent-child mismatch
 - encoded command
 - living-off-the-land usage
-

Step 1 — Generate the activity (endpoint)

On a Windows host with Elastic Agent:

```
cmd.exe /c powershell.exe -enc  
RwBIAHQALQBEAGEAdABIACAafAAgAE8AdQB0AC0ARgBpAGwAZQAgACQAZQBuAHYA  
OgBUAEUATQBQAFwAZABhAHQAZQAUAHQAdAB4AHQACgA=
```

Decoded command:

```
Get-Date | Out-File $env:TEMP\date.txt
```

No malware. Fully explainable.

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\Administrator> cmd.exe /c powershell.exe -enc Rwb1AHQALQBEAGEAdAB1ACAAFAAgAE8AcQB0AC0ARgBpAGwAZQAgACQAZQBUAH
YAOgBUAEUATQBQAFwAZABhAQAZQAuAHQAdAB4AHQACgA=
```

HOW IT WORKS

1 — Execution chain

- **cmd.exe /c**
Launches a command, then exits immediately after it completes. Often used as a launcher.
- **powershell.exe**
Starts PowerShell as a child process of cmd.exe.
- **-enc (EncodedCommand)**
Executes a **Base64-encoded PowerShell command**.
PowerShell expects this to be **UTF-16LE** encoded (important for decoding).

2 — Decode the payload

The Base64 string decodes (UTF-16LE) to:

```
Get-Date | Out-File $env:TEMP\date.txt
```

(There is a trailing newline; functionally irrelevant.)

3 — What the decoded command does

1. **Get-Date**
Retrieves the current system date and time.
2. **Out-File \$env:TEMP\date.txt**
Writes that output to a file named `date.txt` in the user's `%TEMP%` directory, e.g.:
3. `C:\Users\<user>\AppData\Local\Temp\date.txt`

Net effect

- Launches PowerShell via `cmd.exe`
- Hides the real command using Base64
- Collects **current timestamp**
- **Drops a file in %TEMP%**

Step 2 — Go to Discover

In Elastic (Kibana):

Data view

`logs-*`

Time range

Last 15 minutes

Step 3 — Identify the raw events

Filter on:

process.name : "powershell.exe"

Focus fields:

- process.parent.name
- process.command_line
- process.args
- process.executable
- user.name
- host.name

Ignore alerts completely.

Step 4 — Reconstruct the story (this is the test)

You must be able to narrate **in order**:

1. cmd.exe executed interactively
2. cmd.exe spawned powershell.exe
3. PowerShell used -enc
4. Decoded command performed local reconnaissance (date)
5. Output written to user-writable temp path
6. No network activity
7. No follow-on execution

If you cannot tell this story **without looking at detections**, stop.

Step 5 — Analyst decision

Classification matrix

Question	Answer
Is the behavior unusual?	Yes
Is it explainable?	Yes
Is intent clearly malicious?	No
Abuse potential present?	Yes

Correct verdict

Suspicious (controlled, non-malicious)

Anything else is incorrect.

What data you used (explicit)

- **Index:** logs-*
- **Event types:**
 - Process creation (Windows 4688 / endpoint process events)
- **Primary evidence:**
 - process.parent.name
 - process.command_line
 - process.args

No alerts. No rules.

What indicates success

You succeed **only if** you can:

- Explain *why* -enc matters

- Explain *why* cmd → powershell matters
- Explain *why* temp output matters
- Explain *why* this is **not malware**
- Explain *when* the same behavior becomes malicious

If you need a detection rule to justify your answer, you failed.

STAGE 3 VALIDATION — RAW EVENT ANALYSIS (Elastic Security)

1 Generate activity — CONFIRMED

You executed:

```
cmd.exe /c powershell.exe -enc
RwBIAHQALQBEAGEAdABIACAAfAAgAE8AdQB0AC0ARgBpAGwAZQAgACQ
AZQBuAHYAOgBUAEUATQBQAFwAZABhAHQAZQAuAHQAdAB4AHQACgA
=
```

This step is **correct and controlled**.

 message	 A new process has been created. Creator Subject: Security ID: S-1-5-21-479272597-3367404750-1068752215-500 Account Name: Administrator Account Domain: LAB
 process.args	[powershell.exe -enc, RwBIAHQALQBEAGEAdABIACAAfAAgAE8AdQB0AC0ARgBpAGwAZQAgACQAZQBuAHYAOgBUAEUATQBQAFwAZABhAHQAZQAuAHQAdAB4AHQACgA=]
 process.command_line	powershell.exe -enc RwBIAHQALQBEAGEAdABIACAAfAAgAE8AdQB0AC0ARgBpAGwAZQAgACQAZQBuAHYAOgBUAEUATQBQAFwAZABhAHQAZQAuAHQAdAB4AHQACgA=
 process.executable	C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
 process.name	powershell.exe
 process.parent_executable	C:\Windows\System32\cmd.exe

2 Go to Discover — ☒ DONE

You are looking at **raw Windows Security telemetry** ingested into Elastic.

Index:

.ds-logs-system.security-default-*

Event:

event.code : 4688

Correct data source for this stage.

Field	Value
 elastic_agent.snapshot	false
 elastic_agent.version	9.2.4
 event.action	created-process
 event.agent_id_status	verified
 event.category	process
 event.code	4688
 event.created	Feb 6, 2026 @ 11:19:39.123
 event.dataset	system.security
 event.ingested	Feb 6, 2026 @ 11:19:50.000
 event.kind	event

3 Identify raw events — CORRECT EVENT

Core process facts (ground truth)

Field	Value
Event	Process creation
Process	powershell.exe
Parent	cmd.exe
User	Administrator
Command line	powershell.exe -enc <Base64>
Token	Full token (Type 1)

This **matches exactly** what you executed.

No telemetry gaps.

4 Reconstruct the story (this is the real test)

If you can't narrate this cleanly, you fail Stage 3.

Timeline reconstruction

1. User Administrator runs a command in cmd.exe
2. cmd.exe spawns powershell.exe
3. PowerShell is launched with -enc (obfuscated command)
4. Encoded payload decodes to:
5. Get-Date | Out-File \$env:TEMP\date.txt
6. Output is written to a **user-writable temp directory**
7. No network access
8. No child process spawned afterward
9. No persistence mechanisms observed

This reconstruction is **fully supported by raw event data**.

5 Behavioral analysis (SOC reasoning)

What is unusual?

Be precise:

- cmd.exe → powershell.exe (parent–child deviation)
- Use of -enc (intent obfuscation)
- LOLBin usage (powershell.exe)
- Output staged in %TEMP%

None of these are benign *by default*.

When could this be normal?

Valid normal contexts:

- Administrator automation
- Lab testing
- Script wrappers using CMD
- Headless PowerShell execution

Your environment supports this:

- Built-in Administrator
 - Lab hostname
 - No follow-on activity
-

What would make this malicious?

Any of the following would flip the verdict:

- Parent $\neq$ cmd.exe
(e.g. winword.exe, chrome.exe, mshta.exe)
- Encoded payload performs:
 - download
 - execution
 - credential access
- Temp file is later **executed**
- Activity occurs under **standard user**
- Subsequent lateral movement or persistence

None are present here.

Correct SOC verdict

Classification:

Suspicious (controlled, explainable, non-malicious)

This is the *only* correct answer.

Why not benign?

- Attackers use this pattern constantly
- Encoded PowerShell is an abuse signal

Why not malicious?

- Intent is fully observable
 - No attack chain continuation
 - No impact or objective achieved
-

7 Stage-3 success criteria

You have successfully demonstrated:

- Ability to work from **raw 4688 events**
- Correct parent–child interpretation
- Proper handling of encoded PowerShell
- Intent-based classification
- Zero alert dependency

This means **Stage 3 is working as intended**.

8 One critical SOC correction (important)

Do **not** over-trust this line:

SubjectUserName: Administrator

Attackers frequently operate:

- post-exploitation
- with elevated tokens
- inside “trusted” contexts

Privilege ≠ legitimacy.

STAGE 4 — DETECTION ENGINEERING (OPTIONAL BUT POWERFUL)

Duration: 10–14 days

Goal: Understand how EDR detections are **designed, not copied**

Correct detection workflow

1. Observe suspicious behavior manually
2. Describe it in plain English
3. Convert to KQL
4. Validate against historical data
5. Only then → create a rule

What to build (minimum)

- 3 high-quality behavioral detections
- Each with:
 - Clear intent
 - Known false positives
 - Documented investigation steps

This matters more than “50 detections”.

STAGE 5 — RESPONSE DISCIPLINE (REAL SOC MATURITY)

Duration: Ongoing

Goal: Know **when not to respond**

Practice this deliberately

For every suspicious event:

- Should I isolate the host?
- Should I collect more data?
- Should I ignore and document?

Over-response is a **junior analyst mistake**.

STAGE 6 — FAILURE & DEBUGGING (ADVANCED, VERY VALUABLE)

Duration: As needed

Goal: Be the person who fixes EDR, not reinstalls it

Practice breaking and fixing:

- Fleet output issues
- TLS / CA mistakes
- Policy misapplication
- Telemetry drop scenarios

SOC teams value **EDR debuggers** more than rule writers.

FINAL CONCLUSION

This roadmap is not training on Elastic Security.

It is training on **how to think like an EDR analyst once the tool already works.**

By design, it assumes endpoint telemetry is present, reliable, and visible. From that point forward, success or failure in a SOC is no longer determined by agents, dashboards, or alert fidelity—it is determined by **analyst reasoning.**

Across its stages, this roadmap enforces a single outcome:

the ability to explain endpoint behavior accurately, confidently, and without relying on detections.

An analyst who completes this roadmap correctly can:

- Classify endpoint activity directly from raw telemetry
- Separate normal OS and admin behavior from genuine attacker intent
- Explain *why* an event exists, not just *what* fired
- Reconstruct attack chains without alerts, rules, or MITRE labels
- Avoid false escalations, unnecessary isolation, and alert fatigue
- Contribute meaningfully to real incident response, not just triage

Just as importantly, it **filters out false confidence.**

If an analyst cannot pass the exit criteria of each stage without guessing, keyword-matching, or alert dependence, that failure is intentional and informative.

This roadmap produces:

- Fewer detections, but higher-quality ones
- Fewer incidents, but higher-confidence responses
- Analysts who own telemetry instead of reacting to tools

In practical terms, this is the transition from:

"The EDR says it's suspicious"

to

"This behavior is abnormal because of context, intent, and sequence—and here is the evidence."

That shift is the difference between:

- a junior SOC operator
- and a credible EDR analyst

This roadmap exists to make that shift unavoidable.